

PythonIntermediateCourse-es008

June 3, 2026

1 Curso básico de Python

1.1 Apuntes intermedio

Curso básico de Python. Apuntes intermedio por Marcelo Horacio Fortino. Versión 2.5.1. Junio 2026.

Esta obra está sujeta a la licencia Reconocimiento-CompartirIgual 4.0 Internacional de Creative Commons. Para ver una copia de esta licencia, visite <http://creativecommons.org/licenses/by-sa/4.0/>. Puede hallar permisos más allá de los concedidos con esta licencia en <https://fortinux.com>. Sugerencias y comentarios a info@fortinux.com.

Todas las marcas son propiedad de sus respectivos dueños. Python® y PyCon® son marcas registradas de la Python Software Foundation. Linux® es una marca registrada de Linus Torvalds. Ubuntu® es una marca registrada de Canonical Limited. Google® es una marca registrada de Google Inc. Microsoft® y Windows® son marcas registradas de Microsoft Corporation. Red Hat® es una marca comercial registradas de Red Hat, Inc. UNIX® es una marca comercial registrada de The Open Group. Apple® y Mac OS® son marcas comerciales de Apple Inc.

Versión	Autor/es	Fecha	Observaciones
1.0	Marcelo Horacio Fortino	2021/Marzo	Curso Python
1.1	Marcelo Horacio Fortino	2021/Junio	Convertido a markdown - ipynb
1.2	Marcelo Horacio Fortino	2021/Agosto	Actualizados contenidos
1.3	Marcelo Horacio Fortino	2021/Octubre	Agregado Flask microframework
1.4	Marcelo Horacio Fortino	2021/Noviembre	Agregado Pandas - datascience
1.5	Marcelo Horacio Fortino	2021/Diciembre	Agregado Devops - Ansible
2.0	Marcelo Horacio Fortino	2022/Abril	Nueva estructura: core / module
2.1	Marcelo Horacio Fortino	2022/Junio	Módulo apuntes intermedio
2.2	Marcelo Horacio Fortino	2022/Agosto	Actualizado temario y ejercicios

Versión	Autor/es	Fecha	Observaciones
2.3	Marcelo Horacio Fortino	2022/Octubre	Actualizado despliegue a render.com
2.4	Marcelo Horacio Fortino	2023/Noviembre	Aplicaciones de escritorio con python
2.5	Marcelo Horacio Fortino	2026/Abril	Aplicaciones de ML e IA para python

Esta obra se distribuye con la esperanza de que sea útil, pero SIN NINGUNA GARANTÍA, incluso sin la garantía MERCANTIL implícita o sin garantizar la CONVENIENCIA PARA UN PROPÓSITO PARTICULAR. El autor no asume ninguna responsabilidad si el lector hace un mal uso de la misma.

Estos apuntes se basan en: - La documentación oficial de Python, <https://docs.python.org/es/3/tutorial/index.html>, - La bibliografía presentada al final de este documento, y - Documentación propia recogida a lo largo de los años de diversas fuentes.

1.2 Objetivo del curso

- Objetivo general del curso:
- Aprender a usar Python para crear scripts y programas simples plenamente funcionales, desarrollar aplicaciones web, realizar análisis de datos, y utilizar bibliotecas de inteligencia artificial (AI) para aprendizaje automático (ML).
- Objetivos específicos:
- Reconocer las características principales de Python y su utilidad práctica.
- Identificar tipos de datos (simples y compuestos) y operadores.
- Aplicar variables y estructuras de control de flujo.
- Construir funciones y clases (POO).
- Clasificar los módulos y paquetes por sus funcionalidades y objetivos.
- Comparar el lenguaje con otros similares de scripts, procedimentales y orientados a objetos.
- Probar bibliotecas para conexiones REST a aplicaciones web y bases de datos.
- Resolver problemas y errores en el código fuente proponiendo soluciones alternativas (refactoring).
- Utilizar la biblioteca pandas junto con matplotlib y numpy para realizar análisis estadísticos de datos y gráficos.
- Descubrir las herramientas en python de ML y DL (*Deep Learning*) para la inteligencia artificial.
- Como resultado práctico al final del curso cada estudiante habrá creado una aplicación web utilizando el microframework de Python Flask.

1.3 Temario

- Introducción, instalación y compilación
- Datos, expresiones y sentencias
- Variables y funciones, control de flujo

- Clases y objetos, herencia, polimorfismo
- Entradas y salidas con Python
- Gestión de módulos, paquetes y bibliotecas
- Servicios y programas en red, REST API
- Desarrollo de aplicaciones web con Flask
- Análisis de datos con pandas, matplotlib y numpy
- Aprendizaje automático (ML) e inteligencia artificial (AI) con Python
- Módulos opcionales
 - DevOps con Ansible
 - Plotting con Python
 - SDK de GCP para Python
 - SQL ejemplos con pandas
 - Docker y kubernetes con Python
 - Aplicaciones de escritorio con python

1.4 Bibliografía

- Downey, A., Elkner, J., Meyers, C. (2015). Aprenda a Pensar Como un Programador con Python.
Recuperado de <https://argentinaenpython.com/quiero-aprender-python/aprenda-a-pensar-como-un-programador-con-python.pdf>
- Kent D. Lee. (2014). Python, Programming Fundamentals Second Edition.
- Marzal Varó, A., Gracia Luengo, I., García Sevilla, Pedro. (2014). Introducción a la programación con Python 3.
Recuperado de <http://repositori.uji.es/xmlui/handle/10234/102653>
- Miller, B., Ranum, D. Solución de problemas con algoritmos y estructuras de datos usando Python. Traducido por Mauricio Orozco-Alzate, Universidad Nacional de Colombia - Sede Manizales.
Recuperado de <https://runestone.academy/ns/books/published/pythoned/index.html?mode=browsing>
- Shaw, Z. A. (2016). Learn Python 3 the Hard Way.
Recuperado de <https://learnpythonthehardway.org/>
- Van Rossum, G. and the Python development team. (2020). Documentación de Python en español.
Recuperado de https://python-docs-es.readthedocs.io/_/downloads/es/pdf/pdf/
- McGregor, S. E. (2022). Practical Python data wrangling and data quality. O'Reilly Media.
- Chollet, F. (2017). Deep learning with Python. Manning Publications.
- Russell, S., & Norvig, P. (2019). Artificial intelligence: A modern approach. Pearson Higher Education.
- D2L.ai: Interactive Deep Learning Book with Multi-Framework Code, Math, and Discussions.
Recuperado de <https://d2l.ai/>

2 Flask microframework

- Introducción, Crear la aplicación, Estructura, Agregar páginas, Agregar plantillas, Conexión a Github, Despliegue a render.com, Realizar cambios y nuevo despliegue, Recursos, Añadir base de datos, Acceder a las entradas, Crear nuevas entradas, Editar entradas, Eliminar entradas.

2.1 Introducción

- Flask es un framework minimalista escrito en Python que permite crear aplicaciones web rápidamente y con un mínimo número de líneas de código.
- Está basado en:
 - La especificación WSGI de Werkzeug <https://werkzeug.palletsprojects.com/en/stable/>.
 - El motor de templates Jinja <https://jinja.palletsprojects.com/en/stable/>.
- Fuente: <https://flask.palletsprojects.com/en/stable/>.
- En este workshop iremos paso a paso desarrollando una aplicación en Flask para luego hacer el despliegue en la plataforma de aplicaciones cloud de Render <https://www.render.com/>.
- Como requisito se deberá tener o crear una cuenta en esa plataforma y otra cuenta en <https://github.com/> (ambas plataformas tienen una opción gratuita de uso).
- Esto nos permitirá al final del mismo tener una aplicación web en Python disponible en Internet.
- Iniciamos nuestra tarea creando el ambiente virtual:

```
[ ]: # Comandos ejecutados en GNU/Linux
mkdir app
python3 -m venv app/venv
cd app/
source venv/bin/activate
```

```
[ ]: # Comandos ejecutados en Windows
mkdir app
python -m venv app/venv
cd app/venv/Scripts
./activate
```

- Instalamos el microframework Flask y el módulo gunicorn:

```
[ ]: python3 -m pip install flask gunicorn
```

- El fichero *requirements.txt* sirve para instalar módulos de la aplicación si ésta es cambiada de entorno (Sistema operativo, Servidor, etc.).
- Se puede crear manualmente e actualizarlo cuando sea necesario escribiendo los nombres de los módulos a medida que se instalan.
- Como ejemplo, en GNU/Linux creamos el fichero y agregamos *Flask* en el mismo:

```
[ ]: touch requirements.txt
     echo "flask" > requirements.txt
```

- Posteriormente para instalar todos los módulos en el nuevo entorno se ejecutará:

```
[ ]: python3 -m pip install -r requirements.txt
```

- La siguiente opción (alternativa a la anterior y preferible) crea el fichero *requirements.txt* automáticamente.
- Se suele ejecutar una vez completo el desarrollo de la aplicación y previo a la migración al nuevo entorno.

```
[ ]: pip freeze > requirements.txt
```

2.2 Crear la aplicación

- El primer paso es crear el script de ejecución de la aplicación.
- Abrimos un fichero que por convención se lo denomina *app.py* y escribimos el siguiente código modificando el autor y agregando la fecha:

```
[ ]: """
    [Aplicación básica del microframework Flask de Python]

    Author: Fortinux
    Date: []
    """

from flask import Flask

app = Flask(__name__)

@app.route("/")
def index():
    return "<H1>Hola Mundo!</H1>"
```

- Para exportar la variable de ambiente *FLASK_APP* ejecutamos:

```
[ ]: export FLASK_APP=app # Bash de Linux
     set FLASK_APP=app # Terminal de Linux
     $env:FLASK_APP = "app" # Microsoft Powershell
```

- La variable de ambiente para ambientes de producción predeterminada es *production*:

```
[ ]: export FLASK_ENV=production # Bash de Linux
```

- Si queremos habilitar el ambiente de desarrollo para no tener que reiniciar la aplicación cada vez que hacemos cambios en ella, exportamos la variable de ambiente:

```
[ ]: export FLASK_ENV=development # Bash de Linux
      set FLASK_ENV=development # Terminal de Linux
      $env:FLASK_ENV = "development" # Microsoft Powershell
```

- Para ejecutar la aplicación:

```
[ ]: flask run
```

- Se agrega el argumento `-host=0.0.0.0` para publicarlo en la red local.
- Se accede a la aplicación en nuestra máquina desde la URL `127.0.0.1:5000` en el navegador.
- Para acceder desde la red local se utiliza la IP de la máquina donde está alojada la aplicación.

2.3 Estructura

- La estructura de la aplicación será la siguiente:

```
/home/usuario/Proyectos/tutorial_flask
  app.py
  templates/
    base.html
    index.html
|   |   servicios.html
|   |   page_not_found.html
  venv/
  .gitignore
  requirements.txt
```

2.4 Crear páginas

- Para crear las páginas servicios, contacto y admin de la aplicación:

```
[ ]: from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "<H1>Esta es la página principal</H1>"

@app.route("/servicios")
def servicios():
    return "<H1>Esta es la página de servicios</H1>"

@app.route("/contacto")
def contacto():
```

```
return "<H1>Esta es la página de contacto</H1>"
```

2.5 Agregar plantillas

- Para agregar plantillas de diseño web se utilizará *Bootstrap*.
- Bootstrap Front-end open source toolkit
- <https://getbootstrap.com/>.
- Creamos el fichero *index.html* en un nuevo directorio llamado *templates*.
- Esta página HTML será la página principal de la aplicación.

```
[ ]: vim templates/index.html
```

```
[ ]: <!doctype html>
<html>
<head>
  <title>Página principal</title>
  <meta name="description" content="Página principal">
  <meta name="keywords" content="HTML template - plantilla" >
</head>
<body>
  {% extends "base.html" %}
  {% block title %}Página principal{% endblock %}
  {% block content %}
    <h2>Esta es la página principal</h2>

    <main>
      <h1 class="visually-hidden">Heroes examples</h1>
      <div class="px-4 py-5 my-5 text-center">
        
        <h1 class="display-5 fw-bold text-body-emphasis">Centered hero</h1>
        <div class="col-lg-6 mx-auto">
          <p class="lead mb-4">
            Quickly design and customize responsive mobile-first sites with
            Bootstrap, the world's most popular front-end open source toolkit,
            featuring Sass variables and mixins, responsive grid system,
            extensive prebuilt components, and powerful JavaScript plugins.
          </p>
          <div class="d-grid gap-2 d-sm-flex justify-content-sm-center">
            <button type="button" class="btn btn-primary btn-lg px-4 gap-3" >
```

```

        Primary button
    </button>
    <button type="button" class="btn btn-outline-secondary btn-lg px-4">
        Secondary
    </button>
</div>
</div>
</div>
{% endblock %}
</body>
</html>

```

- El siguiente paso es crear una plantilla base para toda la aplicación.
- Tendrá de nombre *base.html* y deberá ser alojada en el directorio *templates* como el resto de las plantillas que se irán agregando a la aplicación.

```
[ ]: vim templates/base.html
```

```
[ ]: <!doctype html>
<html>
<head>
    <title>{% block title %}{% endblock %}</title>
    <meta name="description" content="Página principal" >
    <meta name="keywords" content="html template" >
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.
↪css" rel="stylesheet"
integrity="sha384-sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqukuIAvNWLzeN8tE5YBujZqJLB"
↪crossorigin="anonymous" >
</head>

<body>
<nav class="navbar navbar-expand-lg navbar-light bg-light">
    <div class="container-fluid">
        <a class="navbar-brand" href="https://fortinux.com/">fortinux</a>
        <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
            data-bs-target="#navbarSupportedContent"
↪aria-controls="navbarSupportedContent"
            aria-expanded="false" aria-label="Toggle navigation">
            <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse" id="navbarSupportedContent" >
            <ul class="navbar-nav me-auto mb-2 mb-lg-0">
                <li class="nav-item">
                    <a class="nav-link active" aria-current="page" href="/">Página
↪Principal</a>
                </li>
                <li class="nav-item">

```



```

        <a class="nav-link" href="servicios">Servicios</a>
    </li>
    <li class="nav-item">
        <a class="nav-link" href="contacto">Contacto</a>
    </li>
    <li class="nav-item dropdown">
        <a class="nav-link dropdown-toggle" href="#" id="navbarDropdown"
↪role="button"
        data-bs-toggle="dropdown" aria-expanded="false">Menú desplegable</a>
        <ul class="dropdown-menu" aria-labelledby="navbarDropdown">
            <li><a class="dropdown-item" href="#">Submenú</a></li>
            <li><a class="dropdown-item" href="#">Otro submenú</a></li>
            <li><hr class="dropdown-divider"></li>
            <li><a class="dropdown-item" href="#">Otro enlace</a></li>
        </ul>
    </li>
    <li class="nav-item">
        <a class="nav-link disabled" href="#" tabindex="-1"
↪aria-disabled="true">Deshabilitado</a>
    </li>
</ul>
<form class="d-flex">
    <input class="form-control me-2" type="search" placeholder="Search"
↪aria-label="Search">
    <button class="btn btn-outline-success" type="submit">Buscar</button>
</form>
</div>
</div>
</nav>

```

```

{% block content %}
<h1>Nombre de la organización</h1>
<main>
    <h1 class="visually-hidden">Heroes examples</h1>
    <div class="px-4 py-5 my-5 text-center">
        
        <h1 class="display-5 fw-bold text-body-emphasis">Centered hero</h1>
        <div class="col-lg-6 mx-auto">
            <p class="lead mb-4">
                Quickly design and customize responsive mobile-first sites with

```

```

        Bootstrap, the world's most popular front-end open source toolkit,
        featuring Sass variables and mixins, responsive grid system,
        extensive prebuilt components, and powerful JavaScript plugins.
    </p>
    <div class="d-grid gap-2 d-sm-flex justify-content-sm-center">
        <button type="button" class="btn btn-primary btn-lg px-4 gap-3">
            Primary button
        </button>
        <button type="button" class="btn btn-outline-secondary btn-lg px-4">
            Secondary
        </button>
    </div>
</div>
</div>
</div>
{% endblock %}
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.
    ↪bundle.min.js"
integrity="sha384-FKyoEForCGlyvwx9Hj09JcYn3nv7wiPVlz7YYwJrWVcXK/
    ↪BmnVDxM+D2scQbITxI" crossorigin="anonymous"></script>
</body>
</html>

```

- Creamos la plantilla para la página servicios:

```

[ ]: <!doctype html>
<html>
<head>
    <title>Página servicios</title>
    <meta name="description" content="Página servicios">
    <meta name="keywords" content="HTML template - plantilla" >
</head>

<body>
    {% extends "base.html" %}
    {% block title %}Página Servicios{% endblock %}
    {% block content %}
        <h2>Esta es la página servicios</h2>

        <main>
            <h1 class="visually-hidden">Features examples</h1>
            <div class="container px-4 py-5" id="featured-3" >
                <h2 class="pb-2 border-bottom">Columns with icons</h2>
                <div class="row g-4 py-5 row-cols-1 row-cols-lg-3">
                    <div class="feature col">
                        <div
                            class="feature-icon d-inline-flex align-items-center
                            ↪justify-content-center text-bg-primary bg-gradient fs-2 mb-3">

```

```

        <svg class="bi" width= "1em" height= "1em" aria-hidden="true">
          <use xlink:href="#collection"></use>
        </svg>
      </div>
    <h3 class="fs-2 text-body-emphasis">Featured title</h3>
    <p>
      Paragraph of text beneath the heading to explain the heading.
      We'll add onto it with another sentence and probably just keep
      going until we run out of words.
    </p>
    <a href="#" class="icon-link" >
      Call to action
      <svg class="bi" aria-hidden="true">
        <use xlink:href="#chevron-right"></use>
      </svg>
    </a>
  </div>
  <div class="feature col">
    <div
      class="feature-icon d-inline-flex align-items-center
↳justify-content-center text-bg-primary bg-gradient fs-2 mb-3"
    >
      <svg class="bi" width= "1em" height="1em" aria-hidden="true">
        <use xlink:href="#people-circle"></use>
      </svg>
    </div>
    <h3 class="fs-2 text-body-emphasis">Featured title</h3>
    <p>
      Paragraph of text beneath the heading to explain the heading.
      We'll add onto it with another sentence and probably just keep
      going until we run out of words.
    </p>
    <a href="#" class= "icon-link">
      Call to action
      <svg class="bi" aria-hidden="true">
        <use xlink:href="#chevron-right"></use>
      </svg>
    </a>
  </div>
  <div class="feature col">
    <div
      class="feature-icon d-inline-flex align-items-center
↳justify-content-center text-bg-primary bg-gradient fs-2 mb-3">
      <svg class="bi" width= "1em" height= "1em" aria-hidden="true">
        <use xlink:href="#toggles2"></use>
      </svg>
    </div>
  </div>

```

```

    <h3 class="fs-2 text-body-emphasis">Featured title</h3>
    <p>
        Paragraph of text beneath the heading to explain the heading.
        We'll add onto it with another sentence and probably just keep
        going until we run out of words.
    </p>
    <a href="#" class="icon-link" >
        Call to action
        <svg class="bi" aria-hidden="true">
            <use xlink:href="#chevron-right"></use>
        </svg>
    </a>
</div>
</div>
</div>
{% endblock %}

</body>
</html>

```

- Actualizamos el script ejecutable *app.py*:

```

[ ]: from flask import Flask, render_template

app = Flask(__name__)

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/servicios")
def servicios():
    return render_template("servicios.html")

@app.route("/contacto")
def contacto():
    return render_template("base.html")

```

2.6 Conexión a Github

- Una vez que tenemos nuestra aplicación en Flask, ejecutamos *git init* para inicializar nuestro repositorio.

```
[ ]: git init
```

- Creamos el fichero `.gitignore` para que no suba al servidor git ficheros sensibles.

```
[ ]: venv/  
  
*.pyc  
__pycache__/  
  
instance/  
  
.pytest_cache/  
.coverage  
htmlcov/  
  
dist/  
build/  
*.egg-info/
```

- También podemos utilizar <https://www.toptal.com/developers/gitignore> como base.
- Un ejemplo:

```
[ ]: # Created by https://www.toptal.com/developers/gitignore/api/flask  
# Edit at https://www.toptal.com/developers/gitignore?templates=flask  
### Flask ###  
instance/*  
!instance/.gitignore  
.webassets-cache  
.env  
...  
# Environments  
.venv  
env/  
venv/  
ENV/  
env.bak/  
venv.bak/
```

- Luego se deberá crear un repositorio vacío en Github <https://github.com/>.
- Una vez creado el repositorio, generar un `token` de acceso al repositorio:
- Ir a Settings / Developer settings / Personal access tokens
 - Generate new token
- En el directorio local subiremos los cambios a Github.
- Creamos el commit con todos los cambios hechos hasta ahora en la aplicación:

```
[ ]: git add .  
git commit -m "Primer commit"
```

```
[ ]: # Agregar el repositorio creado en GitHub, por ejemplo
git remote add origin https://github.com/fortinux/nuevoFlask.git
git branch -M main
git push -u origin main
```

2.7 Despliegue a Render

- Ahora se puede hacer un deploy a la plataforma de aplicaciones cloud de Render.
- <https://www.render.com>.
- Esta plataforma ofrece un ambiente cloud en el cual podremos desplegar nuestra aplicación apenas creada.
- Basta crear una cuenta gratuita en <https://dashboard.render.com/register>.
- En <https://www.render.com> clicar en el botón *New*.
 - *Web Service*
- Conectarse al repositorio creado en Github previamente.
- Escribir un nombre para el subdominio.
- Los parámetros de configuración son los siguientes:
 - Environment Python
 - Build Command `pip install -r requirements.txt`
 - Start Command `gunicorn app:app`
- Al guardar los cambios iniciará el despliegue.
- Si el *Web Service* creado se ha guardado con el nombre *nuevo*, la aplicación se podrá ver en la dirección web:
- <https://nuevo.onrender.com/>

2.8 Realizar cambios y nuevo despliegue

- Para agregar una página que gestione los errores HTML 404 (página no encontrada) se crea un nuevo decorador en *app.py*:

```
[ ]: @app.errorhandler(404)
def page_not_found(error):
    return render_template('page_not_found.html'), 404
```

- Luego se crea la página *page_not_found.html* en el directorio *templates*:

```
[ ]: # Copiamos la página index.html
cp templates/index.html templates/page_not_found.html
# Editamos la página page_not_found.html
vim templates/page_not_found.html
```

```
[ ]: # Hacemos el commit y lo subimos a github
git add app.py templates/page_not_found.html
git commit -m "Agregada página 404"
git push -u origin main
```

- Render <https://render.com/> hará automáticamente el nuevo despliegue.

2.9 Recursos

- Configurar un ambiente virtual
<https://mothergeo-py.readthedocs.io/en/latest/development/how-to/venv-win.html>
- Flask Quickstart
<https://flask.palletsprojects.com/en/latest/quickstart/>
- Bootstrap Front-end open source toolkit
<https://getbootstrap.com/>
- Crear un fichero .gitignore para Git
<https://www.toptal.com/developers/gitignore>
- Documentación Render para el microframework Flask
<https://render.com/docs/deploy-flask>

2.10 Añadir base de datos

- El siguiente paso es crear una base de datos donde poder guardar las entradas en la aplicación.
- Para ello programamos un fichero con el *schema* de la base de datos (*schema.sql*) y otro con la consulta para la creación de la misma (*crear_db.py*).

```
[ ]: # Fichero schema.sql
DROP TABLE IF EXISTS posts;

CREATE TABLE posts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    creado TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    titulo TEXT NOT NULL,
    contenido TEXT NOT NULL
);
```

```
[ ]: # Fichero crear_db.py
import sqlite3

connection = sqlite3.connect('database.db')

with open('schema.sql') as f:
    connection.executescript(f.read())

cur = connection.cursor()

cur.execute("INSERT INTO posts (titulo, contenido) VALUES (?, ?)",
            ('Primera entrada', 'Contenido de la primera entrada: Hola mundo!'))
```

```

    )

cur.execute("INSERT INTO posts (titulo, contenido) VALUES (?, ?)",
            ('Segunda entrada', 'Contenido de la segunda entrada: bla, bla'))

connection.commit()
connection.close()

```

- Ejecutamos el script:

```
[ ]: python crear_db.py
```

- El siguiente paso es editar *app.py* para conectarnos a la base de datos y obtener las entradas:

```
[ ]: # Fichero app.py
from flask import Flask, render_template
import sqlite3

app = Flask(__name__)

def get_db_connection():
    conn = sqlite3.connect('database.db')
    conn.row_factory = sqlite3.Row
    return conn

@app.route("/")
def home():
    conn = get_db_connection()
    posts = conn.execute('SELECT * FROM posts').fetchall()
    conn.close()
    return render_template("index.html", posts=posts)

...

```

- Editamos la plantilla HTML (*templates/index.html*) para reflejar los cambios:

```
[ ]: <!doctype html>
<html>
<head>
  <title>Página principal</title>
  <meta name="description" content="Página principal">
  <meta name="keywords" content="HTML template - plantilla" >
</head>

```



```

<body>
    {% extends "base.html" %}
    {% block title %}Página principal{% endblock %}
    {% block content %}
        <h2>Esta es la página principal</h2>
        <p>Un párrafo de bienvenida.</p>
        {% for post in posts %}
            <a href="#">
                <h2>{{ post['titulo'] }}</h2>
            </a>
            <button type="button" class="btn btn-primary">{{ post['creado'] }}</
→button>
            <hr>
        {% endfor %}
    {% endblock %}
</body>
</html>

```

2.11 Acceder a las entradas

- Para acceder a las entradas se deberá crear la ruta a las mismas y utilizar una función que devuelva una página de error 404 en el caso que no existan.
- Los ficheros a editar son *app.py* y una nueva plantilla *templates/entrada.html*.

```

[ ]: # Fichero app.py
from flask import Flask, render_template
import sqlite3
from werkzeug.exceptions import abort

app = Flask(__name__)

def get_db_connection():
    conn = sqlite3.connect('database.db')
    conn.row_factory = sqlite3.Row
    return conn

def get_post(post_id):
    conn = get_db_connection()
    post = conn.execute('SELECT * FROM posts WHERE id = ?',
                        (post_id,)).fetchone()

    conn.close()
    if post is None:
        abort(404)
    return post

```

```
...
```

```
@app.route('/<int:post_id>')
def post(post_id):
    post = get_post(post_id)
    return render_template('entrada.html', post=post)
```

```
[ ]: # Fichero template/entrada.html
{% block content %}
    <h2>{% block title %} {{ post['titulo'] }} {% endblock %}</h2>
    <button type="button" class="btn btn-primary">{{ post['created'] }}</
    ↪button>
    <p>{{ post['contenido'] }}</p>
{% endblock %}
```

2.12 Crear nuevas entradas

- Para la creación de nuevas entradas se utilizará *requests* y las funciones de *flask*:
 - *url_for()* para generar URLs.
 - *flash()* para mostrar mensajes al usuario.
 - *redirect()* para cambiar de página.

```
[ ]: # Fichero app.py
# Se agregan las funciones de flask
from flask import Flask, render_template, request, url_for, flash, redirect
import sqlite3
from werkzeug.exceptions import abort

app = Flask(__name__)
app.config['SECRET_KEY'] = 'clave'

...
```

- Luego se deberá crear una nueva función para ingresar nuevas entradas a la base de datos.

```
[ ]: # Fichero app.py

...

@app.route('/crear', methods=('GET', 'POST'))
def crear():
    if request.method == 'POST':
        title = request.form['title']
        content = request.form['content']
```

```

    if not title:
        flash('Se requiere un título para la entrada')
    else:
        conn = get_db_connection()
        conn.execute('INSERT INTO posts (titulo, contenido) VALUES (?, ?)',
                      (title, content))
        conn.commit()
        conn.close()
        return redirect(url_for('home'))

return render_template('crear.html')

```

- Por último, se crea una nueva plantilla con el formulario necesario para ingresar nuevas entradas.

```

[ ]: # Fichero templates/crear.html
{% extends 'base.html' %}

{% block content %}
<h1>{% block title %} Crear una nueva entrada {% endblock %}</h1>
<form method="post">
    <div class="form-group">
        <label for="title">Título</label>
        <input type="text" name="title"
              placeholder="Título de la entrada" class="form-control"
              value="{ request.form['title'] }"></input>
    </div>

    <div class="form-group">
        <label for="content">Contenido</label>
        <textarea name="content" placeholder="Contenido"
                  class="form-control">{ request.form['content'] }</textarea>
    </div>
    <div class="form-group">
        <button type="submit" class="btn btn-primary">Enviar</button>
    </div>
</form>
{% endblock %}

```

2.13 Editar entradas

- Para editar las entradas se necesita crear una nueva función que ejecute un *UPDATE* en la base de datos.

```

[ ]: # Fichero app.py
@app.route('/<int:id>/editar', methods=('GET', 'POST'))
def editar(id):
    post = get_post(id)

```

```

if request.method == 'POST':
    title = request.form['title']
    content = request.form['content']

    if not title:
        flash('Se requiere un título para la entrada')
    else:
        conn = get_db_connection()
        conn.execute('UPDATE posts SET titulo = ?, contenido = ?'
                     ' WHERE id = ?',
                     (title, content, id))
        conn.commit()
        conn.close()
        return redirect(url_for('home'))

return render_template('editar.html', post=post)

```

- Luego se creará una plantilla nueva con el formulario necesario para editar las entradas.

```

[ ]: # Fichero templates/editar.html
{% extends 'base.html' %}

{% block content %}
<h1>{% block title %} Edit "{{ post['titulo'] }}" {% endblock %}</h1>

<form method="post">
    <div class="form-group">
        <label for="title">Título</label>
        <input type="text" name="title" placeholder="Título de la entrada"
              class="form-control"
              value="{{ request.form['titulo'] or post['titulo'] }}">
        </input>
    </div>

    <div class="form-group">
        <label for="content">Contenido</label>
        <textarea name="content" placeholder="Contenido de la entrada"
              class="form-control">{{ request.form['contenido'] or
post['contenido'] }}</textarea>
    </div>
    <div class="form-group">
        <button type="submit" class="btn btn-primary">Enviar</button>
    </div>
</form>
<hr>
{% endblock %}

```

- Finalmente se actualiza la plantilla *index.html* con el enlace para editar las entrada.

```
[ ]: # Fichero templates/index.html
<!doctype html>
<html>
<head>
  <title>Página principal</title>
  <meta name="description" content="Página principal">
  <meta name="keywords" content="HTML template - plantilla">
</head>

<body>
  {% extends "base.html" %}
  {% block title %}Página principal{% endblock %}
  {% block content %}
  <h2>Esta es la página principal</h2>
  <p>Un párrafo de bienvenida.</p>
  {% for post in posts %}
    <a href="{% url_for('post', post_id=post['id']) %}">
      <h2>{{ post['titulo'] }}</h2>
    </a>
    <a href="{% url_for('editar', id=post['id']) %}">
      <button type="button" class="btn btn-primary">Editar</button>
    </a>
    <button type="button" class="btn btn-primary">{{ post['creado'] }}</
    <button>
      <hr>
    {% endfor %}
  {% endblock %}
</body>
</html>
```

2.14 Eliminar entradas

- Para eliminar entradas se creará el metodo *delete* en el fichero *app.py*.

```
[ ]: # Fichero app.py
...

@app.route('/<int:id>/delete', methods=('POST',))
def delete(id):
    post = get_post(id)
    conn = get_db_connection()
    conn.execute('DELETE FROM posts WHERE id = ?', (id,))
    conn.commit()
    conn.close()
    flash("{} ha sido eliminado".format(post['titulo']))
    return redirect(url_for('home'))
```

- Finalmente se agregará un botón para eliminar la entrada en la plantilla *editar.html*.

```
[ ]: # vim templates/editar.html
...

<form action="{% url_for('delete', id=post['id']) %}" method="POST">
    <input type="submit" value="Eliminar entrada"
          class="btn btn-danger btn-sm"
          onclick="return confirm('¿Quieres eliminar esta entrada?')">
</form>
<hr>
{% endblock %}
```